

LM -> Language Model

=> understands and predict text

types of lm

-> traditional lm

-> neural lm

LLM -> large language model

LLM = predicts next word → repeatedly → forms intelligent output

foundation model -> llm

more parameter -> more complex the model

llm -> data+architecture+training

transformer

pretraining => RLHF (reinforcement learning with human feedback)

fine tuning

GPT => Generative Pre-trained Transformer

Transformer:

input => tokens => vectors => attention => MLP => repeated layers

Tensor => weights => outputs

weights define the model

Embedding matrix (W_e) maps tokens to vectors

Softmax converts logits to probabilities

Neural Network:

neurons hold values (activations)

learning = finding weights and biases

gradient descent used

Prompt Engineering:

programming in natural language

task, role, format, constraints

few-shot prompting

Vector Database:

embedding = array of numbers

semantic search

RAG:

1. Retrieval
2. Augmentation
3. Generation

Chunking:

size + overlap

Fine-tuning:

customize model for specific tasks

MCP server:

connects LLM with tools and APIs